Export System

This document describes how the export system works, the algorithms used to transform sign-in data into reports, and the libraries involved. The dashboard offers three output formats -- Excel (.xlsx), PDF, and CSV -- all built from the exact same session data, described below in PDF and CSV Formats; everything up to that section describes the original Excel pipeline that the other two formats are built on top of.

Overview

The export system generates formatted Excel (.xlsx) files from sign-in records. It produces two types of reports:

1. **Member Export** -- Member sign-in/out data.
2. **Staff Export** -- Staff sign-in/out data.

Both reports use the **same layout**: one worksheet per person, with one row per actual sign-in/out session (nobody -- member or staff -- is restricted to a fixed AM/PM pattern; anyone can sign in and out any number of times, any time of day), a duration per session, and a running total per person. The only difference between the two is the title text ("Kiosk Sign-In Export" vs. "Kiosk Staff Sign-In Export") -- everything else, including the underlying grouping/pairing logic and the per-sheet layout function, is shared.

> **Why they're unified:** member and staff exports used to look completely different -- members got a per-calendar-day sheet with a fixed AM/PM grid, while staff got a single sheet with one page-broken section per person. That made the two exports inconsistent to read, and put everyone for a bulk staff download on one sheet with page breaks between them rather than separate sheet tabs. Both were changed to the one-worksheet-per-person, one-row-per-session format described below so member and staff exports are visually identical and every bulk download gives each person their own sheet tab.

Either report can also be generated for a single person (see Single-Person Export below), reusing the exact same layout function.

The export logic lives in `kiosk_signin/export_util.py` and uses the `openpyxl` library for spreadsheet generation and `Pillow` (PIL) for rendering signature images.

Shared Person Export

Data Flow

1. `database.get_signs_for_export(date_from, date_to)` (members) or `database.get_staff_signs_for_export(date_from, date_to)` (staff) retrieves the raw sign events.
2. `generate_export()` / `generate_staff_export()` are thin wrappers that both call the shared `_generate_person_sessions_export(signs, date_from, date_to, output_path, title_prefix)`, passing a different `title_prefix` string.

3. `_build_staff_sessions(signs)` groups events by person (member or staff -- the function itself doesn't care which) and pairs them into sessions. Despite the `staff_` in its name (left over from before the two exports were unified), it's the shared grouping function for both.
4. `_generate_person_sessions_export()` creates one worksheet per person (title = that person's name, made Excel-safe and de-duplicated by `_unique_sheet_title()`) and calls `_write_person_sheet()` to populate each one.

Session Pairing Algorithm (`_build_staff_sessions`)

Events for each person are sorted chronologically, then walked through with a simple alternation state machine -- the first `IN` opens a session, the next `OUT` closes it:

```
for e in events:
   if e["direction"] == "IN":
       if open_in is not None:
           sessions.append({"date": open_in["timestamp"][:10], "in": open_in, "out": None})
       open_in = e
   else:  # OUT
       if open_in is not None:
           sessions.append({"date": open_in["timestamp"][:10], "in": open_in, "out": e})
           open_in = None
       else:
           sessions.append({"date": e["timestamp"][:10], "in": None, "out": e})
if open_in is not None:
   sessions.append({"date": open_in["timestamp"][:10], "in": open_in, "out": None})
```

This handles the edge cases that unrestricted sign-in/out opens up:

- **Normal session**: an `IN` followed by an `OUT` -- both recorded, duration calculated.
- **Still signed in**: a session with no closing `OUT` yet (e.g. exporting today's data before someone has signed out, or a genuinely forgotten sign-out -- see Filling In a Missing Sign-In/Sign-Out for how that gets corrected) -- kept with `out: None` and shown as "Still signed in" in the sheet, with the duration cell left blank and excluded from the person's total.
- **Stray `OUT` with nothing open**: kept with `in: None` so the data isn't silently dropped.
- **A second `IN` while one is already open**: closes the previous session with no sign-out (defensive -- shouldn't normally happen, since the kiosk auto-detects the next expected direction) and opens a new one.

The function returns a list of `(person_key, sessions)` tuples sorted by last name, where `person_key = (member_id, first_name, last_name, role)` -- `role` is empty for members, since only staff have a role/position field.

Duration Calculation

The `_duration_mins()` function calculates the time between two sign events in minutes:

```
def _duration_mins(start_event, finish_event):
   t1 = datetime.strptime(start_event["timestamp"], "%Y-%m-%d %H:%M:%S")
   t2 = datetime.strptime(finish_event["timestamp"], "%Y-%m-%d %H:%M:%S")
   mins = int((t2 - t1).total_seconds() / 60)
```

```
    return mins if mins >= 0 else None
```

Durations are formatted as "Xh YYm" (e.g. "3h 45m") by the `_fmt_duration()` helper. Each person's total ("Total time in the centre: Xh YYm") sums every session's duration across the whole exported date range -- sessions with no duration (still open, or missing a sign-in/out) simply contribute nothing.

Date and Year Formatting

Each row's date is shown as day/month only (`_day_month()`, e.g. "08/06") -- the year isn't repeated on every row. Instead, `_year_label()` computes the year (or year range, e.g. "2026–2027", if the exported data spans a calendar year boundary) once per person and shows it in their name banner.

Sheet Titles (`_unique_sheet_title`)

Excel worksheet titles can't contain `[ ] : * ? / \` and are capped at 31 characters. `_unique_sheet_title(label, used)` strips any of those characters out of a person's name, truncates to 31 characters, and de-duplicates against every title already used in the same workbook by appending " (2)", " (3)", etc. (truncating further if needed to keep the whole title within 31 characters) -- so two people with the same name after sanitising still each get their own tab.

Spreadsheet Layout

Each person's worksheet has this structure:

```
Row 1:    Title bar (merged across all columns) -- e.g. "Kiosk Staff Sign-In Export | 2026-06-01 to 2026
Row 2:    Name banner (merged, purple background) -- "Name -- Role -- Year" (role omitted for members)
Row 3:    Column headers (Date, Sign In, Sign Out, Duration)
Row 4+:   One row per session, in chronological order:
            Date | Sign In time | Sign Out time (or "Still signed in") | Duration
          followed by a second row underneath -- only when at least one
          signature was captured for that session -- with the signature
          image(s) placed under the Sign In / Sign Out columns.
Row N:     "Total time in the centre: Xh YYm" (merged, bold)
```

If a person has no sign-in records at all in the exported range, their sheet shows "No sign-in records for this period." instead of a row list. If the whole export has no data at all (no people matched the range), a single sheet is created with that same message instead of the workbook being empty.

Single-Person Export

Both the web dashboard and the kiosk admin panel can export just one person's sign-in/out records instead of the whole roster.

Data Flow

1. `database.get_signs_for_member(member_id, date_from=None, date_to=None)` retrieves every sign event for one person. With no dates it returns their full history; with `date_from`/`date_to` it restricts to that range using the same `DATE(s.timestamp) >= ?` / `<= ?` filtering as the bulk export queries. Its SELECT includes `sign_ins.id` and `sign_ins.notes` alongside the fields the export functions need, so the same query can also feed `dashboard/app.py`'s `_pair_sign_in_visits()` for the "My Timesheet" self-service page (see 06-flask-web-dashboard.md) without a second query.
2. The caller checks the person's `person_type` and picks the matching wrapper function -- `person_type == "staff"` → `generate_staff_export()`, otherwise → `generate_export()`. Since both now delegate to the same shared renderer, this only changes the title text on the sheet, not the layout. The same `person_type` check picks between the PDF and CSV variants of these wrappers (see PDF and CSV Formats).

Entry Points

- **Dashboard:** `GET /export/person?person_id=<id>&date_from=<optional>&date_to=<optional>&fmt=<optional>` (see `dashboard/app.py:export_person`). Date params are optional -- omitting them exports the person's entire history. `fmt` is `xlsx` (default), `pdf`, or `csv`. Reachable from the "Export Sign-in History" button on a person's edit page, and the "Export a Person's Sign-ins" card on the Exports page (which lets an admin pick any member or staff person, a date range, and a format from a form).
- **Dashboard, staff only:** `GET /export/staff?person_id=<id>&date_from=<optional>&date_to=<optional>&fmt=<optional>` (see `dashboard/app.py:export_staff`) does the same single-person export as above, restricted to staff, from the "Staff Signing Sheets" card's own person dropdown -- so staff exports don't require going to the generic person picker. Both routes share the single-person logic via an `_export_person_response(person, date_from, date_to, fmt)` helper in `app.py`, which builds the file in the requested format and flashes a "no records" error if the range is empty.
- **Dashboard, self-service:** `GET /my-timesheet/download?date_from=<optional>&date_to=<optional>&fmt=<optional>` lets a logged-in staff member download their own history through the same `_export_person_response()` helper -- the person is always resolved from their session, never from a query parameter (see 09-authentication-and-security.md for why that matters).
- **Kiosk admin panel:** the "Export" button next to "Edit" in the Member Management and Staff Management search results opens a small date-range dialog (`_ExportRangeDialog` in `kiosk_signin/screens/admin_screen.py`) with optional From/To fields, Today/7-day/30-day quick-select buttons, and an "All history" button that clears the range. Confirming calls `AdminPanelScreen._export_single_person()`, which now accepts optional `date_from`/`date_to` and passes them straight through to `database.get_signs_for_member()`. Leaving both fields blank preserves the original full-history behaviour; the saved filename gains a `_<date_from>_to_<date_to>` suffix whenever a range is given. The file still saves to the user's home directory, same as the bulk kiosk exports. The kiosk only ever produces `.xlsx` -- the PDF/CSV formats are dashboard-only.

Bulk Exports on the Dashboard

The dashboard's Exports page only has a bulk (whole-roster, date-range) option for staff, via `GET /export/staff` with no `person_id` -- this calls `generate_staff_export()` (or its PDF/CSV equivalent, depending on `fmt`) directly over `database.get_staff_signs_for_export()`, rather than through `_export_person_response()` (which is single-person only). There is no dashboard-side bulk export for all members -- exporting members individually via `/export/person` is the only dashboard path. The kiosk admin panel's own bulk member export (a separate feature, using `database.get_signs_for_export()` and `generate_export()` directly from `kiosk_signin/screens/admin_screen.py`) is unaffected, still exists, and only ever produces `.xlsx`.

PDF and CSV Formats

Every export function described above has a PDF and a CSV counterpart, all defined in `kiosk_signin/export_util.py` next to the `.xlsx` versions, and all built from the same `_build_staff_sessions(signs)` grouping/pairing output described in Session Pairing Algorithm -- the three formats never diverge on *what* data they show, only on how it's rendered:

Excel	PDF	CSV
`generate_export()`	`generate_export_pdf()`	`generate_export_csv()`
`generate_staff_export()`	`generate_staff_export_pdf()`	`generate_staff_export_csv()`

CSV

`_generate_person_sessions_csv()` writes one flat sheet with the stdlib `csv` module -- CSV has no concept of separate worksheets, so every person's sessions live in the same file, separated by a per-person "Total" row instead of a sheet break:

```
writer.writerow(["Name", "Role", "Date", "Sign In", "Sign Out", "Duration"])
...
writer.writerow([name, role, _day_month(sess["date"]), in_text, out_text, _fmt_duration(mins)])
...
writer.writerow([name, role, "", "", "Total", _fmt_duration(total_mins)])
```

The dashboard writes to an `io.StringIO()` buffer (the `csv` module is text-mode), then encodes it to bytes for `send_file()`, which expects a binary stream:

```
text_buf = io.StringIO()
generate_export_csv(signs, export_from, export_to, text_buf)
buf = io.BytesIO(text_buf.getvalue().encode("utf-8"))
```

PDF

`_generate_person_sessions_pdf()` renders a small HTML document -- one `<div>` per person, each with a `Date | Sign In | Sign Out | Duration` `<table>` matching the Excel layout -- and converts it with `xhtml2pdf.pisa.CreatePDF(html, dest=output)`. Every person after the first starts on a new page via a CSS `page-break-before: always;` rule on their `<div>`, mirroring the Excel version's one-sheet-per-person separation.

`xhtml2pdf` was already a project dependency used by `Docs/generate_printable_docs.py` to turn the markdown docs into PDFs -- reused here rather than adding a second PDF library. Since it's now a runtime feature (not just a docs-generation dev tool), it was moved from being an implicit dev-only dependency into `requirements.txt`, and the PyInstaller build in `.github/workflows/build.yml` gained `--hidden-import xhtml2pdf --collect-submodules xhtml2pdf --collect-data xhtml2pdf` (the `--collect-data` flag matters -- xhtml2pdf ships font/data files that PyInstaller's static analysis wouldn't otherwise find) so PDF export works in the packaged `.exe`, not just when running from source.

Names and roles are HTML-escaped (`html.escape()`) before being interpolated into the generated markup, since they're free-text fields that could otherwise contain characters that break the HTML (or, in principle, get interpreted as markup) -- the Excel and CSV paths don't need this since `openpyxl`/`csv` write cell values directly, with no markup layer in between.

Signature Rendering

Signatures are stored in the database as serialised Python lists of stroke coordinates `[(x1, y1, x2, y2), ...]`. The `render_signature()` function (public -- it's reused outside this module, see Last Signature Display below) converts these into PNG images:

1. Deserialise the byte string back into a list using `ast.literal_eval()`.
2. Calculate the bounding box of all stroke coordinates.
3. Scale the strokes to fit within a target image size (160x60 pixels) with margins.
4. Create a white PIL `Image` and draw each stroke as a line.
5. Save the image to a `BytesIO` buffer as PNG.
6. Return the PNG bytes for embedding in the spreadsheet.

```
img = Image.new("RGB", (width, height), "white")
draw = ImageDraw.Draw(img)
for (x1, y1, x2, y2) in strokes:
   draw.line([(tx(x1), ty(y1)), (tx(x2), ty(y2))], fill="black", width=2)
```

The scaling uses `min()` to maintain the signature's aspect ratio:

```
scale = min((width - margin * 2) / range_x, (height - margin * 2) / range_y)
```

The rendered PNGs are embedded into Excel cells using `openpyxl.drawing.image.Image`.

Last Signature Display

Outside of exports, both the kiosk and the dashboard can show a member or staff person's most recently used signature without generating a full spreadsheet:

- `database.get_last_signature(member_id)` returns the most recent `sign_ins` row for that person that has a **non-null, non-empty** signature -- it deliberately skips rows with no signature (e.g. records from before signatures were made mandatory), unlike `get_last_sign()` which returns the most recent row regardless of whether it has a signature.
- Like the rest of the "current state" functions in `database.py` (`get_last_sign`, `get_today_last_sign`, `get_allowed_direction`), this is computed live from `sign_ins` on every call -- there is no separate stored/cached "current signature" column anywhere in the schema.
- **Kiosk:** `ProfileScreen` and `StaffProfileScreen` each have a "Last Signature on File" section, populated in `_populate_signature()`, which renders the signature via `export_util.render_signature()` and displays it as an image with a "Last used: `<timestamp>` (`<direction>`)" caption. If the person has no sign-in history, or none of their sign-ins have a captured signature, a "No signature on file yet." placeholder is shown instead.
- **Dashboard:** the person edit page (`dashboard/templates/person_form.html`) shows the same signature as an `<img>` tag pointing at `GET /people/<id>/signature.png` (`dashboard/app.py:person_signature`), an `@admin_required` route that streams the rendered PNG or returns a 404 with an empty body if there's nothing to show. The `<img>` tag's `onerror` handler swaps in "No signature on file yet." text client-side when the route 404s.

Output Handling

Both wrapper functions accept an `output_path` parameter that can be either a file path string (used by the kiosk's admin panel) or a file-like object (used by the dashboard's in-memory export). This is possible because `openpyxl`'s `Workbook.save()` method accepts both.

Print Setup

Every sheet is configured for A4 portrait printing, fit to page width (one page wide, unlimited height), with custom page margins (0.4 inch left/right, 0.6 inch top/bottom). A person with a lot of sessions in the exported range can spill onto extra printed pages by themselves -- that's expected, not an error.